DevOps

DevOps, yazılım geliştirme alanında sıkça kullanılan bir terimdir. Ancak sektörde yeni de olsan veya yıllardır sektörde de olsan DevOps'un ne olduğunu ve çalışma şeklini nasıl etkileyebileceğini bilmen gerekir. Bu metinde DevOps'u keşfedecek ve kültürü ile ortak bileşenlerini inceleyeceksin.

DevOps nedir?

DevOps'un tanımı, kime sorduğuna bağlıdır! Ancak çoğu uzmana göre DevOps şu şekilde özetlenebilir:

> ***DevOps**, yazılım geliştirme (Dev) ve operasyon (Ops) ekiplerinin hızlı ve yüksek kaliteli yazılım güncellemeleri sunmak için işbirliği yaptığı bir yazılım geliştirme yaklaşımıdır.*

Bu tanımın temel bileşenlerini analiz edelim.

Geliştirme

DevOps'taki "Dev", **geliştirme** ekibini ifade eder. Bu ekip; yazılımın kodunu yazar, düzenler, belgelendirir ve bunları bir kod tabanında ya da havuzunda saklar.

BT operasyonları

DevOps'taki "Ops", **bilişim teknolojisi (BT) operasyonları** ekibini ifade eder. Bu ekip; donanım, yazılım, ağ ve bulut depolama dahil olmak üzere geliştirmenin dayalı olduğu altyapıyı kurar, yönetir ve optimize eder. Ekip ayrıca yazılım güncellemelerini kullanıcılara dağıtır ve başka şekillerde kodu destekler.

Hız

DevOps'un hedeflerinden biri, bir projenin parçası olan ürün veya hizmetlerin hızlı dağıtma yoluyla hızlı ve sık bir şekilde yayınlanmasıdır. Kuruluşlar, DevOps ile büyük ve seyrek değişiklikler yerine küçük ama sürekli güncellemeler yayınlar.

Kalite

Çıkışların yüksek kalitede olması, yani paydaşların performans, işlevsellik, istikrar, güvenilirlik, kullanılabilirlik ve güvenlik beklentilerini karşılaması gerekir. Bu paydaşlar arasında geliştirme ve operasyon ekiplerinin yanı sıra güvenlik ekipleri, iş kolu ekipleri ve kullanıcılar da bulunmaktadır.

Kültür

DevOps yalnızca kuruluş bir DevOps kültürü benimserse işe yarar ve bu kültürün köşe taşı da **işbirliğidir.**

Geliştirme ve operasyon ekipleri geleneksel olarak kendi silolarında ayrı ayrı çalışırlar. Ancak DevOps ile **yazılım geliştirme yaşam çevrimi (SDLC)** boyunca birlikte çalışır ve açık iletişimi sürdürürler. Bilgiyi özgürce ve şeffaf bir şekilde paylaşıp geliştiricilerin herhangi bir yazılım sorunu hakkında **hızlı geribildirim** almasını sağlamak için birlikte çalışırlar.

Ekipler ayrıca ürün kalitesiyle ilgili **sorumluluğu da paylaşır**.

- Geliştiriciler hem kodu oluşturur hem de test eder ve zaman içinde bakımını yapar. Bu görevleri operasyon ekibine bırakmazlar.
- Operasyon personeli ise yazılım güncellemeleriyle ilgili planlama görüşmelerine katılır. Böylece geliştiricilerin sistemin gerektirdiği izleme, günlüğe kaydetme, güvenlik testi ve diğer özellikleri oluşturmalarını sağlayabilirler.

Ekipler, SDLC boyunca işbirliği yaparak birbirlerinin ihtiyaçlarını daha iyi anlar. Buna karşılık çalışma uygulamalarını bu ihtiyaçları karşılayacak şekilde ayarlayabilir, ekip uyumunu iyileştirebilir ve yayınlamayı geciktiren engelleri ortadan kaldırabilirler.

Bileşenler

DevOps'un yaygın olarak tanınan diğer bileşenlerinden bahsedelim.

Çeviklik

DevOps genellikle yazılım geliştirmeye yönelik çevik bir yaklaşım içerir. Çevik yaklaşımı açıklamak için öncelikle onun öncülü olan şelale yaklaşımını ele alalım.

Şelale yaklaşımı doğrusal bir yöntemdir. Ekip; planlama, tasarım, geliştirme, test etme ve destek gibi belirgin ardışık aşamalardan oluşan uzun ve ayrıntılı bir plan uygular. Bu plan, her aşamayla ilgili son tarihleri ve her ekip üyesinin sorumluluklarını belirler ve amaç, planı mümkün olduğunca kesin bir şekilde uygulamaktır.

Ancak bu yaklaşım, yazılım geliştirme açısından çeşitli sorunlar ortaya çıkarmaktadır.

- Proje ilerledikçe geliştiriciler engellerle karşılaşabilir veya yeni, daha iyi fikirler düşünebilir ve müşteriler gereksinimlerini değiştirebilir. Ancak katı planlar, uyum sağlamayı zor ve bazen maliyetli hale getirir.
- Daha da kötüsü müşteriler ve kullanıcılar genellikle ürünü test aşamasına kadar göremedikleri için geliştiriciler o zamana kadar geri bildirim alamazlar. Hataları ve

müşteri endişelerini sürecin sonlarına doğru ele almak, geliştirme sürecinin erken aşamalarına göre daha maliyetli ve zaman alıcı olabilir.

Buna karşılık **çevik geliştirme** yinelemeli bir süreçtir. Projeyi daha uzun, ayrı aşamalarda tamamlamak yerine, ekipler 'Sprint' olarak bilinen ve genellikle her biri birkaç hafta süren kısa yinelemeler halinde çalışır. Her sprint'in amacı, müşterinin hızlıca geri bildirimde bulunabileceği küçük bir teslim edilebilir ürün üretmektir. Bu şekilde, geliştiriciler müşterilerin endişelerini erkenden öğrenebilir ve bunları dikkate alarak ilerleyebilir. Ayrıca, testler proje zaman çizelgesinin sonlarına doğru değil, her sprint boyunca gerçekleştirildiği için ekipler hataları daha erken tanımlayıp düzeltebilir.

Şelale projesi gibi çevik projenin de genel bir hedefi vardır. Ancak plan daha az detaylı ve yeni fikir ve gereksinimlere uyum sağlayacak şekilde daha esnektir.

Kısacası, çevik yaklaşım DevOps için idealdir.

- Kısa sprint'lere, küçük teslimlere ve sürekli testlere odaklanmak DevOps'un sık ve yüksek kaliteli yazılım güncellemeleri hedefiyle uyumludur.
- Geliştiriciler ve operasyon personeli, değişen proje taleplerine ve kalite standartlarına hızlı bir şekilde uyum sağlayabilir.

Mikro hizmetler

DevOps, dağıtım hızını artırmak için mikro hizmet odaklı bir uygulama tasarımına ihtiyaç duyar. **Mikro hizmet**, diğer bileşenlerle birlikte çalışarak tam uygulamayı oluşturan, bağımsız olarak işlev görebilen küçük bir uygulama bileşenidir.

Geleneksel geliştirme sürecinde geliştiriciler bir uygulamanın tüm parçalarını büyük, tek bir parça halinde oluşturur. Bir özellik eklerken veya bir hatayı düzeltirken geliştiricilerin tüm uygulamayı yeniden devreye almaları gerekir.

Ancak mikro hizmetleri kullanırken geliştiricilerin uygulamanın tamamını değil, yalnızca ilgili bileşeni düzenlemesi gerekir. Bu küçük çaplı değişiklikler, güncellemeleri daha hızlı hale getirir ve uygulamanın bozulma riskini azaltır.

Otomasyon

DevOps, kodun SDLC boyunca hareketini **otomatikleştirmek** için araçların kullanılmasını içerir.

Örneğin test yapmayı ele alalım. Kod, yaşam döngüsü boyunca farklı seviyelerde testlerden geçmelidir ve çeşitli araçlar her aşama için uygun testleri otomatik olarak çalıştırabilir. Araçlar ayrıca altyapı bileşenlerini hazırlayabilir ve yapılandırabilir, performans ve güvenlik sorunlarını izleyebilir, uyumsuz bileşenleri veya varlıkları tanımlayabilir ve kodu üretime dağıtabilir.

Otomasyon, devreye alma süresini ve insan hatası olasılığını azaltır. Ayrıca geliştiricilerin ve operasyon personelinin kodlama, problem çözme ve ürünü iyileştirmek için diğer önemli görevlere odaklanmasını sağlar.

Sürekli entegrasyon ve sürekli teslim

Sürekli entegrasyon ve sürekli teslim (CI/CD), DevOps için vazgeçilmez iki otomatik süreçtir.

- **Sürekli bütünleştirme (CI)**, geliştiricilerin yeni yazılmış kodun küçük parçalarını düzenli olarak ana kod tabanlarına entegre ettikleri bir süreçtir. Kod otomatik teste tabi tutular ve kod tabanı yöneticisi, entegrasyondan önce kodu inceleyip onaylar.
- **Sürekli teslim (CD)** daha sonra gelir. Bu, yeni kodun üretim öncesi bir ortama gönderilerek üretime hızlı ve güvenli bir şekilde dağıtılmaya hazır olduğunun doğrulandığı bir süreçtir. Burada kod ek testlerden geçirilir, incelenir ve ardından üretim aşamasına geçer.

CI/CD, DevOps için paha biçilmezdir.

- Testleri otomatikleştirir, kodu manuel olarak test etmek için harcanan zamandan tasarruf sağlar ve geliştiricilere hatalar hakkında anında geri bildirim sağlar.
- **DevSecOps**'un (güvenliğin yerleşik olduğu DevOps) standart parçaları olan uygulama ve saldırıya açıklık taraması gibi siber güvenlik kontrollerini otomatikleştirir.
- Sık test, tarama ve inceleme aynı zamanda daha yüksek kaliteli kod anlamına gelir.
- CI/CD, ön üretim ortamına dağıtımı otomatikleştirir.

CI/CD ile güncellemeler, daha hızlı ve daha az çabayla üretime ulaşır.